

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

ОТЧЁТ
о лабораторной работе № 1
«Генерация бинарного кода Грея.
Мультимножества и операции над ними»
по дисциплине «Дискретная математика»

Выполнил

студент гр. 5130201/50302

Р. Ю. Дужнов

Проверил

преподаватель

А. В. Востров

«____» _____ 2026 г.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Математическое описание	4
1.1 Мультимножества и операции над ними	4
1.2 Универсум на коде Грея	5
1.3 Генерация бинарного кода Грея	5
1.4 Случайное заполнение мультимножества заданной мощности	6
2 Особенности реализации	7
2.1 Средства разработки и структура программы	7
2.2 Ограничения	7
2.3 Исключения	8
2.4 Случайные числа	9
2.5 Генерация кода Грея и носитель	9
2.6 Класс Multiset и построение универсума	11
2.7 Модуль ввода	13
2.8 Класс Menu и функция main	13
3 Результаты работы	15
3.1 Сценарий 1: случайная кратность, автоматическое заполнение	15
3.2 Сценарий 2: ручной ввод с ошибками и отмена	17
3.3 Сценарий 3: граничные мощности A и B	20
3.4 Сценарий 4: некорректный ввод команд и параметров	21
3.5 Сценарий 5: разрядность $n = 0$	22
3.6 Сценарий 6: пересоздание универсума и конец ввода	24
3.7 Сценарий 7: наибольшая разрядность $n = 19$	25
ЗАКЛЮЧЕНИЕ	27
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	28
ПРИЛОЖЕНИЕ А Текст программы	29

ВВЕДЕНИЕ

Требуется реализовать программу, которая генерирует бинарный код Грея заданной пользователем разрядности n и строит на нём универсум мультимножеств U . На основе U формируются мультимножества A и B ; для каждого пользователь выбирает способ заполнения (вручную или автоматически) и задаёт мощность. Программа выводит результаты операций: объединение, пересечение, разность, симметрическая разность, дополнение, арифметические сумма, разность, произведение и деление. Требуется меню и защита от некорректного ввода.

Принято: все элементы U имеют одинаковую кратность $k \geq 1$ (вводится или выбирается случайно); $A, B \subseteq U$, поэтому $0 \leq |A|, |B| \leq |U|$; разность, дополнение, арифметические разность и деление вычисляются в обоих вариантах, всего результатов тринадцать. Входные данные — в таблице 1.

Таблица 1 — Входные данные программы

Величина	Допустимые значения	Пустая строка
Команда меню	одна из букв <code>u a b t o m h q</code> (регистр не важен)	ошибка
Разрядность n	целое от 0 до 19	ошибка
Кратность k	целое от 1 до 100	случайное k
Мощность $ A , B $	целое от 0 до $ U $	случайная мощность
Способ заполнения	<code>r</code> (случайное) или <code>m</code> (ручное)	отмена
Строка ручного заполнения	пары код : кратность или код через пробел	отмена

1 Математическое описание

1.1 Мультимножества и операции над ними

Мультимножество A над конечным множеством X (носителем) — совокупность элементов X , в которую элемент x входит $k_A(x) \geq 0$ раз [1]; $k_A: X \rightarrow \{0, 1, 2, \dots\}$ — функция кратности. Запись: $A = \{x_1^{a_1}, \dots, x_s^{a_s}\}$, элементы с нулевой кратностью не записываются. Мощность — сумма кратностей:

$$|A| = \sum_{x \in X} k_A(x). \quad (1)$$

Универсум U — мультимножество над X с $k_U(x) \geq 1$. A — подмультимножество U ($A \subseteq U$), если

$$k_A(x) \leq k_U(x) \quad \text{для всех } x \in X. \quad (2)$$

Операции выполняются поэлементно [1, 2] (таблица 2). Все кратности в таблице не превышают $k_U(x)$ (сумма и произведение ограничены явно), поэтому результат любой операции — подмультимножество U .

Таблица 2 — Операции над мультимножествами

Операция	Обозначение	Кратность x в результате
Объединение	$A \cup B$	$\max\{k_A(x), k_B(x)\}$
Пересечение	$A \cap B$	$\min\{k_A(x), k_B(x)\}$
Дополнение	$\bar{A}$	$k_U(x) - k_A(x)$
Разность	$A \setminus B = A \cap \bar{B}$	$\min\{k_A(x), k_U(x) - k_B(x)\}$
Симметрическая разность	$A \triangle B$	$\max\{\min\{k_A(x), k_U(x) - k_B(x)\}, \min\{k_B(x), k_U(x) - k_A(x)\}\}$
Арифметическая сумма	$A + B$	$\min\{k_A(x) + k_B(x), k_U(x)\}$
Арифметическая разность	$A - B$	$\max\{k_A(x) - k_B(x), 0\}$
Арифметическое произведение	$A * B$	$\min\{k_A(x) \cdot k_B(x), k_U(x)\}$
Арифметическое деление	A/B	$\lfloor k_A(x)/k_B(x) \rfloor$ при $k_B(x) > 0$; 0 при $k_B(x) = 0$

Вычисляются $A \cup B, A \cap B, A \setminus B, B \setminus A, A \triangle B, \bar{A}, \bar{B}, A + B, A - B, B - A,$

$A * B, A/B, B/A$. Для контроля программа проверяет следствия определений

$$|\bar{A}| = |U| - |A|, \quad |A \cup B| + |A \cap B| = |A| + |B|. \quad (3)$$

1.2 Универсум на коде Грея

Носитель X — все 2^n двоичных слов длины n в порядке кода Грея; i -й элемент — i -е слово кода. Кратности всех элементов равны:

$$k_U(x) = k, \quad 1 \leq k \leq K_{\max} = 100, \quad |U| = k \cdot 2^n. \quad (4)$$

При $n = 0$ носитель считается пустым (слово длины 0 нельзя ни вывести, ни ввести): $U = \{\}$, $|A| = |B| = 0$, все результаты пусты.

1.3 Генерация бинарного кода Грея

Бинарный код Грея порядка n — последовательность всех 2^n двоичных слов длины n , в которой соседние слова, а также последнее и первое, различаются ровно в одном разряде [1].

Алгоритм [1]: слово — битовая шкала $B = (B[n-1], \dots, B[0])$, на шаге i инвертируется разряд с номером

$$Q(i) = \text{число нулей в конце двоичной записи } i. \quad (5)$$

Разряды нумеруются с 0, как биты числа; в [1] — с 1, поэтому там номер на 1 больше.

Вход: целое $n \geq 1$. **Выход:** 2^n шкал длины n .

1. $B[j] := 0$ для $j = 0, \dots, n-1$; выдать B .

2. Для $i = 1, \dots, 2^n - 1$: $p := Q(i)$; $B[p] := 1 - B[p]$; выдать B .

При $0 < i < 2^n$ выполняется $0 \leq Q(i) \leq n-1$. Трудоёмкость — $O(2^n)$.

Протокол при $n = 3$ — в таблице 3.

Таблица 3 — Протокол алгоритма при $n = 3$

Шаг i	Двоичная запись i	$p = Q(i)$	Шкала B
—	—	—	000
1	001	0	001
2	010	1	011
3	011	0	010
4	100	2	110
5	101	0	111
6	110	1	101
7	111	0	100

1.4 Случайное заполнение мультимножества заданной мощности

Задача: по ограничению V (в программе $V = U$) и числу $m \in [0, |V|]$ построить случайное $A \subseteq V$, $|A| = m$. Элемент x свободен, если $k_A(x) < k_V(x)$.

Вход: V, m . **Выход:** $A \subseteq V$, $|A| = m$.

1. Если $m > |V|$ — ошибка.
2. $k_A(x) := 0$ для всех x ; список F — все x с $k_V(x) > 0$.
3. m раз: выбрать равновероятно позицию j в F , $x := F[j]$, $k_A(x) := k_A(x) + 1$; если $k_A(x) = k_V(x)$, записать на место j последний элемент F и укоротить F на 1.

Каждый шаг выбирает только свободный элемент, поэтому холостых выборов нет. Трудоёмкость — $O(2^n + m)$.

2 Особенности реализации

2.1 Средства разработки и структура программы

Язык C++14, только стандартная библиотека; компилятор GCC 6.3.0 (MinGW, 32-разрядная сборка), ОС Windows 11. Сборка — CMake (файл CMakeLists.txt, цели lab1 и bench) или командой

```
g++ -std=c++14 -O2 -Isrc src/main.cpp src/ui/*.cpp  
src/core/*.cpp -o lab1
```

Слой core — математическая часть без консоли, слой ui — ввод и меню; ui зависит от core, но не наоборот (таблица 4). Текст программы — в приложении А.

Таблица 4 — Файлы программы

Файл	Назначение
src/core/GrayCode.hpp, .cpp	генерация кода Грея, класс CodeSet (носитель)
src/core/Multiset.hpp, .cpp	класс Multiset, операции, случайное заполнение, вывод
src/core/Universe.hpp, .cpp	построение универсума
src/core/Random.hpp, .cpp	генератор случайных чисел
src/ui/Input.hpp, .cpp	защищённый построчный ввод
src/ui/Menu.hpp, .cpp	класс Menu: состояние и меню
src/main.cpp	точка входа, ключ --seed
tools/bench.cpp	замер времени

2.2 Ограничения

$n_{\max} = 19$ (MAX_DEPTH): любое действие должно выполняться быстрее 1 с. Самое долгое — случайное заполнение при $|A| = |U| = K_{\max} \cdot 2^n$. Замер tools/bench.cpp (Intel Core i7-13700KF, -O2, лучший из трёх запусков) — в таблице 5: при $n = 19$ время 0,53 с, при $n = 20$ — 1,58 с.

Таблица 5 — Время действий, мс (наихудший случай, $K_{\max} = 100$)

n	Код Грея	Универсум	Заполнение A	13 операций
16	0,0	0,0	45	1,0
17	0,0	0,0	95	4,1
18	0,5	0,5	239	7,0
19	1,0	1,0	531	15
20	2,0	2,0	1581	53
21	8,5	6,5	12 368	67

$K_{\max} = 100$ (MAX_MULTIPPLICITY) выбрано вместе с $n_{\max}$: от него зависит время заполнения. Все величины хранятся в `size_t`; наибольшие значения ($|U| = 52\,428\,800$, произведение кратностей 10 000) не переполняют его и в 32-разрядной сборке.

Вывод: таблица — не больше 32 строк, слова кода — не больше 16, запись мультимножества при $n \leq 4$ — полностью, иначе $\max\{4, \lfloor 100/(n+6) \rfloor\}$ элементов; матрица — столько столбцов, сколько помещается в 112 символов. Скрытая часть отмечается пометкой; мощности всегда считаются по всем элементам.

2.3 Исключения

Ошибки пользователя исключений не вызывают: выводится сообщение и запрос повторяется. Исключения (таблица 6) означают ошибку в программе или конец ввода.

Таблица 6 — Классы исключений

Класс	Родитель	Условие
GrayCodeError	<code>std::runtime_error</code>	общий родитель ошибок кода Грея
InvalidDepthError	GrayCodeError	$n > \text{MAX_TYPE_DEPTH}$
CodeIndexError	GrayCodeError	номер кода вне носителя
MultisetError	<code>std::runtime_error</code>	общий родитель ошибок мультимножеств
MultisetIndexError	MultisetError	номер элемента вне носителя
CarrierMismatchError	MultisetError	операнды над разными носителями
NotSubmultisetError	MultisetError	операнд не подмультимножество U
CardinalityOverflowError	MultisetError	мощность больше мощности ограничения
InvalidMultiplicityError	MultisetError	$k \notin [1, K_{\max}]$
InputClosedError	<code>std::runtime_error</code>	поток ввода закончился

2.4 Случайные числа

Генератор — `std::mt19937` с `std::uniform_int_distribution` (`std::rand` не подходит: $\text{RAND_MAX} = 32767$ меньше числа элементов при $n \geq 16$).

- `make_seed()` — зерно: `std::random_device` XOR текущее время;
- `seed_random(seed)` — задать зерно;
- `random_below(bound)` — равновероятное целое из $[0, \text{bound} - 1]$.

Запуск `lab1 --seed N` повторяет все случайные числа.

2.5 Генерация кода Грея и носитель

Шкала хранится целым числом; `changed_bit(i)` вычисляет $Q(i)$ по (5), инвертирование — `code ^= 1 << changed_bit(i)` (листинг 1). `generate_gray_code(depth)` возвращает 2^n кодов (пусто при $n = 0$), `code_to_string(code, depth)` — строку из n символов 0 и 1.

Листинг 1 — Генерация кода Грея (`GrayCode.cpp`)

```

4 // Функция, определяющая, какой бит перевернуть
5 static size_t changed_bit(size_t i){
6     size_t q = 0;
7     size_t j = i;
8     while(j % 2 == 0){
9         j /= 2;
10        q++;
11    }
12    // число нулей в конце двоичной записи i — номер бита справа, считая с
    нуля
13    return q;
14 }
15
16 std::vector<size_t> generate_gray_code(size_t depth){
17     /*
18         Алгоритм начинает с 0 и на каждом шаге переворачивает
19         один бит, какой бит перевернуть — решает функция changed_bit()
20     */
21     if(depth > MAX_TYPE_DEPTH){
22         throw InvalidDepthError();
23     }
24     std::vector<size_t> codes;
25     // при n = 0 кодов нет
26     if(depth == 0){
27         return codes;
28     }
29     size_t count = size_t(1) << depth;
30     codes.reserve(count);
31     size_t code = 0;
32     codes.push_back(code);
33     for(size_t i = 1; i < count; i++){
34         size_t p = changed_bit(i);
35         code ^= size_t(1) << p;
36         codes.push_back(code);
37     }
38     return codes;
39 }

```

Класс `CodeSet` — носитель: поля `depth` (n), `codes` (i -й код) и `positions` (номер кода по его значению, поиск за $O(n)$). Методы — в таблице 7.

Таблица 7 — Методы класса `CodeSet`

Метод	Вход	Выход
<code>CodeSet(depth)</code>	n	носитель из 2^n кодов
<code>get_depth()</code>	—	n
<code>size()</code>	—	2^n (0 при $n = 0$)
<code>get_code(i)</code>	номер	строка кода; <code>CodeIndexError</code>
<code>find(code, index)</code>	строка	<code>true</code> и номер, если строка — слово длины n

2.6 Класс `Multiset` и построение универсума

`Multiset` хранит общий носитель (`std::shared_ptr<const CodeSet> carrier`) и кратности по номерам элементов (`std::vector<size_t> counts`). Этим классом представлены U , A , B и все результаты; каждая операция выполняется за $O(2^n)$ без поиска элементов. Методы — в таблицах 8 и 9; операции не меняют операнды и возвращают новый объект.

Таблица 8 — Методы доступа класса `Multiset`

Метод	Вход	Выход
<code>Multiset(carrier)</code>	носитель	нулевые кратности
<code>size()</code>	—	число элементов носителя
<code>get_count(i), set_count(i, c)</code>	номер, кратность	чтение и запись; <code>MultisetIndexError</code>
<code>cardinality()</code>	—	мощность (1)
<code>is_submultiset_of(other)</code>	мультимножество	выполнено ли (2)

Таблица 9 — Методы операций класса Multiset

Метод	Результат	Реализация
<code>unite(b)</code>	$A \cup B$	<code>max</code>
<code>intersect(b)</code>	$A \cap B$	<code>min</code>
<code>complement(u)</code>	$\overline{A}$	$k_U(x) - k_A(x)$
<code>difference(b, u)</code>	$A \setminus B$	<code>intersect</code> и <code>complement</code>
<code>symmetric_difference(b, u)</code>	$A \triangle B$	<code>difference</code> и <code>unite</code>
<code>arithmetic_sum(b, u)</code>	$A + B$	$\min\{k_A(x) + k_B(x), k_U(x)\}$
<code>arithmetic_difference(b)</code>	$A - B$	$\max\{k_A(x) - k_B(x), 0\}$
<code>arithmetic_product(b, u)</code>	$A * B$	$\min\{k_A(x) \cdot k_B(x), k_U(x)\}$
<code>arithmetic_division(b)</code>	A/B	$\lfloor k_A(x)/k_B(x) \rfloor, 0$ при $k_B(x) = 0$

`random_submultiset(bound, cardinality)` реализует алгоритм подраздела 1.4 (листинг 2). `print_multiset(out, multiset, limit)` выводит запись вида $\{000 \times 2, 011 \times 1\}$, пустое — $\{\}$. `uniform_universe(carrier, k)` строит U с кратностями k .

Листинг 2 — Случайное заполнение мультимножества (Multiset.cpp)

```

135 Multiset random_submultiset(const Multiset& bound, size_t cardinality){
136     if(cardinality > bound.cardinality()){
137         throw CardinalityOverflowError();
138     }
139     Multiset res(bound.carrier);
140     // номера элементов, кратность которых в res ещё меньше, чем в bound
141     std::vector<size_t> free;
142     free.reserve(bound.counts.size());
143     for(size_t i = 0; i < bound.counts.size(); i++){
144         if(bound.counts[i] > 0) free.push_back(i);
145     }
146     // cardinality раз: случайный элемент со свободным местом получает +1 к
    кратности;
147     // список не пустеет раньше времени, так как свободных мест |bound| -
    step > 0
148     for(size_t step = 0; step < cardinality; step++){
149         size_t j = random_below(free.size());
150         size_t i = free[j];
151         res.counts[i]++;
152         if(res.counts[i] == bound.counts[i]){
153             // элемент заполнен: на его место в списке встает последний,
            список короче на 1
154             free[j] = free.back();
155             free.pop_back();

```

```

156         }
157     }
158     return res;
159 }

```

2.7 Модуль ввода

Ввод читается построчно (`std::getline`) и проверяется целиком; оператор `>>` не используется, так как принимает «12abc» как 12 и «-1» как большое беззнаковое число. При вводе из файла прочитанная строка выводится на экран.

- `read_text(prompt)` — строка без пробелов по краям; при конце потока — `InputClosedError`;
- `parse_number(text, value)` — строка из 1–9 цифр;
- `is_code(text, depth)` — строка из n символов 0 и 1;
- `read_number(prompt, min, max), read_number_or_empty(...)` — число из $[min, max]$ (второй вариант допускает пустую строку); сообщение называет причину отказа;
- `read_letter(prompt, letters, allow_empty)` — одна буква из `letters`, регистр не важен.

2.8 Класс Menu и функция main

Menu хранит носитель, U, A, B и признаки их готовности. Перед каждым приглашением «>» выводится строка со списком команд (таблица 10); ошибки начинаются с «!». Недоступная команда не выполняется, а сообщение называет, чего не хватает.

Таблица 10 — Команды меню

Буква	Действие	Метод	Условие
u	создать универсум	<code>create_universe</code>	—
a, b	задать A, B	<code>fill_multiset</code>	U создан
t	таблица кратностей U, A, B	<code>show_table</code>	U создан
o	13 операций и проверка (3)	<code>show_operations</code>	заданы A, B
m	матрица кратностей	<code>show_matrix</code>	заданы A, B
h	справка: формулы, ограничения	<code>show_help</code>	—
q	выход	—	—

Создание универсума: читаются n и k (при $n = 0$ кратность не запрашивается); выводятся код Грея, U и $|U|$. Прежние A и B сбрасываются.

Заполнение: читаются мощность m и способ; заполняется копия, поэтому при отмене прежнее мультимножество сохраняется. Ручной ввод — строки пар код:кратность (код без кратности — 1), пока $|A| < m$. Строка применяется целиком или отвергается с причиной, если код не из n символов 0 и 1, кратность не целое ≥ 1 , кратность больше свободного места кода $k_U(x) - k_A(x)$ (с учётом той же строки) или сумма больше остатка $m - |A|$. Так сохраняются $A \subseteq U$ и $|A| = m$.

Функция main включает вывод UTF-8 в консоли Windows, задаёт зерно и запускает меню; `InputClosedError` — штатное завершение («ввод закончился», код 0), прочие исключения — код 1.

3 Результаты работы

Программа проверена на семи сценариях: ввод — в `report/scenarios/*.txt`, запуск — `lab1 --seed N < файл`, N — номер сценария, вывод воспроизводим. Рисунки построены из протоколов скриптом `report/make_figures.py`.

3.1 Сценарий 1: случайная кратность, автоматическое заполнение

Запуск и справка — рисунки 1, 2.

```
seed = 1

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> h
```

Рисунок 1 — Запуск программы, зерно 1

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> h
запись {000×2, 011×1}: код Грея × кратность, {} - пустое; kU, kA, kB - кратности в U, A, B
A ∪ B max(kA, kB) объединение
A ∩ B min(kA, kB) пересечение
A \ B min(kA, kU - kB) разность
B \ A min(kB, kU - kA) разность
A Δ B max(k[A\B], k[B\A]) симметрическая разность
¬A kU - kA дополнение
¬B kU - kB дополнение
A + B min(kA + kB, kU) арифметическая сумма
A - B max(kA - kB, 0) арифметическая разность
B - A max(kB - kA, 0) арифметическая разность
A * B min(kA * kB, kU) арифметическое произведение
A / B kA div kB; 0 при kB = 0 арифметическое деление
B / A kB div kA; 0 при kA = 0 арифметическое деление
0 <= n <= 19, 1 <= k <= 100, 0 <= |A|, |B| <= |U|
```

Рисунок 2 — Справка (команда h)

Универсум $n = 3$, k выбрана случайно: $k = 42$, слова кода — в порядке таблицы 3, $|U| = 8 \cdot 42 = 336$ (рисунок 3).

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 3
k [1..100, Enter - случайно]:
k = 42
код Грея, 2^3 = 8: 000 001 011 010 110 111 101 100
U = {000×42, 001×42, 011×42, 010×42, 110×42, 111×42, 101×42, 100×42}, |U| = 336
```

Рисунок 3 — Создание универсума: $n = 3$, случайная кратность $k = 42$

$|A| = 150$ задана, $|B| = 219$ выбрана случайно (рисунок 4); в таблице все $k_A(x), k_B(x) \leq 42$ (рисунок 5).

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..336, Enter - случайно]: 150
r - случайно, m - вручную, Enter - отмена: r
A = {000×21, 001×20, 011×14, 010×21, 110×21, 111×16, 101×14, 100×23}, |A| = 150

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..336, Enter - случайно]:
|B| = 219
r - случайно, m - вручную, Enter - отмена: r
B = {000×37, 001×30, 011×28, 010×20, 110×31, 111×20, 101×28, 100×25}, |B| = 219

```

Рисунок 4 — Случайное заполнение: $|A|$ задана, $|B|$ выбрана случайно

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код  kU  kA  kB
000  42  21  37
001  42  20  30
011  42  14  28
010  42  21  20
110  42  21  31
111  42  16  20
101  42  14  28
100  42  23  25
Σ    336 150 219

```

Рисунок 5 — Таблица кратностей U, A, B

Результаты операций и матрица — рисунки 6, 7; проверка (3): $186 = 336 - 150$, $220 + 149 = 150 + 219$. Ручной расчёт (таблица 11) совпадает со столбцами 000, 010, 100 матрицы.

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {000×37, 001×30, 011×28, 010×21, 110×31, 111×20, 101×28, 100×25}, |A ∪ B| = 220
A ∩ B = {000×21, 001×20, 011×14, 010×20, 110×21, 111×16, 101×14, 100×23}, |A ∩ B| = 149
A \ B = {000×5, 001×12, 011×14, 010×21, 110×11, 111×16, 101×14, 100×17}, |A \ B| = 110
B \ A = {000×21, 001×22, 011×28, 010×20, 110×21, 111×20, 101×28, 100×19}, |B \ A| = 179
A Δ B = {000×21, 001×22, 011×28, 010×21, 110×21, 111×20, 101×28, 100×19}, |A Δ B| = 180
¬A    = {000×21, 001×22, 011×28, 010×21, 110×21, 111×26, 101×28, 100×19}, |¬A| = 186
¬B    = {000×5, 001×12, 011×14, 010×22, 110×11, 111×22, 101×14, 100×17}, |¬B| = 117
A + B = {000×42, 001×42, 011×42, 010×41, 110×42, 111×36, 101×42, 100×42}, |A + B| = 329
A - B = {010×1}, |A - B| = 1
B - A = {000×16, 001×10, 011×14, 110×10, 111×4, 101×14, 100×2}, |B - A| = 70
A * B = {000×42, 001×42, 011×42, 010×42, 110×42, 111×42, 101×42, 100×42}, |A * B| = 336
A / B = {010×1}, |A / B| = 1
B / A = {000×1, 001×1, 011×2, 110×1, 111×1, 101×2, 100×1}, |B / A| = 9
проверка: |¬A| = |U| - |A|: 186 = 336 - 150 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 220 + 149 = 150 + 219 верно

```

Рисунок 6 — Результаты тринадцати операций (сценарий 1)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> m

	000	001	011	010	110	111	101	100	·
U	42	42	42	42	42	42	42	42	336
A	21	20	14	21	21	16	14	23	150
B	37	30	28	20	31	20	28	25	219
$A \cup B$	37	30	28	21	31	20	28	25	220
$A \cap B$	21	20	14	20	21	16	14	23	149
$A \setminus B$	5	12	14	21	11	16	14	17	110
$B \setminus A$	21	22	28	20	21	20	28	19	179
$A \Delta B$	21	22	28	21	21	20	28	19	180
$\neg A$	21	22	28	21	21	26	28	19	186
$\neg B$	5	12	14	22	11	22	14	17	117
$A+B$	42	42	42	41	42	36	42	42	329
$A-B$	0	0	0	1	0	0	0	0	1
$B-A$	16	10	14	0	10	4	14	2	70
$A*B$	42	42	42	42	42	42	42	42	336
A/B	0	0	0	1	0	0	0	0	1
B/A	1	1	2	0	1	1	2	1	9

Рисунок 7 — Матрица кратностей (сценарий 1)

Таблица 11 — Ручная проверка (сценарий 1, $k_U = 42$)

	000: $k_A=21, k_B=37$	010: $k_A=21, k_B=20$	100: $k_A=23, k_B=25$
$A \cup B, A \cap B$	37, 21	21, 20	25, 23
$A \setminus B$	$\min\{21, 5\} = 5$	$\min\{21, 22\} = 21$	$\min\{23, 17\} = 17$
$B \setminus A$	$\min\{37, 21\} = 21$	$\min\{20, 21\} = 20$	$\min\{25, 19\} = 19$
$A \Delta B$	$\max\{5, 21\} = 21$	$\max\{21, 20\} = 21$	$\max\{17, 19\} = 19$
$\overline{A}, \overline{B}$	21, 5	21, 22	19, 17
$A + B$	$\min\{58, 42\} = 42$	$\min\{41, 42\} = 41$	$\min\{48, 42\} = 42$
$A - B, B - A$	0, 16	1, 0	0, 2
$A * B$	$\min\{777, 42\} = 42$	$\min\{420, 42\} = 42$	$\min\{575, 42\} = 42$
$A/B, B/A$	$\lfloor 21/37 \rfloor = 0,$ $\lfloor 37/21 \rfloor = 1$	1, 0	0, 1

3.2 Сценарий 2: ручной ввод с ошибками и отмена

Кратности 0, 101 и «abc» отвергаются, принято $k = 2, |U| = 8$ (рисунок 8).

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 2
k [1..100, Enter - случайно]: 0
! 0 не входит в [1..100]
k [1..100, Enter - случайно]: 101
! 101 не входит в [1..100]
k [1..100, Enter - случайно]: abc
! «abc» не целое неотрицательное число; нужно число из [1..100]
k [1..100, Enter - случайно]: 2
код Грея, 2^2 = 4: 00 01 11 10
U = {00×2, 01×2, 11×2, 10×2}, |U| = 8

```

Рисунок 8 — Ввод кратности универсума с ошибками

Ручное заполнение A мощности 4 (рисунок 9): отвергаются «0», «0a:1» (не код), «00:3» (свободно 2), «00:x» (не число), «00:2 01 10 11» (5 единиц при остатке 4, строка не применяется целиком) и повторный «00». Итог: $A = \{00^2, 11^2\}$.

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..8, Enter - случайно]: 4
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
A = {}, осталось 4: 0
! «0»: код должен состоять из 2 символов 0 и 1
A = {}, осталось 4: 0a:1
! «0a:1»: код должен состоять из 2 символов 0 и 1
A = {}, осталось 4: 00:3
! «00:3»: у 00 свободно 2 (kU = 2, занято 0)
A = {}, осталось 4: 00:x
! «00:x»: после двоеточия нужна кратность - целое число от 1
A = {}, осталось 4: 00:2 01 10 11
! в строке 5 ед., а осталось распределить 4
A = {}, осталось 4: 00:2
A = {00×2}, осталось 2: 00
! «00»: у 00 свободно 0 (kU = 2, занято 2)
A = {00×2}, осталось 2: 11:2
A = {00×2, 11×2}, |A| = 4

```

Рисунок 9 — Ручное заполнение A с ошибками ввода

Ручное заполнение B отменено пустой строкой, затем $B = \{00^2, 11^1\}$ заполнено случайно (рисунок 10). Результаты — рисунки 11, 12.

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..8, Enter - случайно]:
|B| = 3
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
B = {}, осталось 3:
отменено

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..8, Enter - случайно]: 3
r - случайно, m - вручную, Enter - отмена: r
B = {00×2, 11×1}, |B| = 3

```

Рисунок 10 — Отмена ручного заполнения B и случайное заполнение

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {00×2, 11×2}, |A ∪ B| = 4
A ∩ B = {00×2, 11×1}, |A ∩ B| = 3
A \ B = {11×1}, |A \ B| = 1
B \ A = {}, |B \ A| = 0
A Δ B = {11×1}, |A Δ B| = 1
¬A = {01×2, 10×2}, |¬A| = 4
¬B = {01×2, 11×1, 10×2}, |¬B| = 5
A + B = {00×2, 11×2}, |A + B| = 4
A - B = {11×1}, |A - B| = 1
B - A = {}, |B - A| = 0
A * B = {00×2, 11×2}, |A * B| = 4
A / B = {00×1, 11×2}, |A / B| = 3
B / A = {00×1}, |B / A| = 1
проверка: |¬A| = |U| - |A|: 4 = 8 - 4 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 4 + 3 = 4 + 3 верно

```

Рисунок 11 — Результаты операций (сценарий 2)

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> m

```

	00	01	11	10	·
U	2	2	2	2	8
A	2	0	2	0	4
B	2	0	1	0	3
A ∪ B	2	0	2	0	4
A ∩ B	2	0	1	0	3
A \ B	0	0	1	0	1
B \ A	0	0	0	0	0
A Δ B	0	0	1	0	1
¬A	0	2	0	2	4
¬B	0	2	1	2	5
A + B	2	0	2	0	4
A - B	0	0	1	0	1
B - A	0	0	0	0	0
A * B	2	0	2	0	4
A / B	1	0	2	0	3
B / A	1	0	0	0	1

Рисунок 12 — Матрица кратностей (сценарий 2)

3.3 Сценарий 3: граничные мощности A и B

$n = 3, k = 2, |U| = 16$ (рисунок 13); $|A| = 17$ отвергается, $|A| = 16$ даёт $A = U, |B| = 0 \text{ — } B = \{\}$ (рисунок 14).

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 3
k [1..100, Enter - случайно]: 2
код Грея, 2^3 = 8: 000 001 011 010 110 111 101 100
U = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |U| = 16
```

Рисунок 13 — Универсум $n = 3, k = 2$

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..16, Enter - случайно]: 17
! 17 не входит в [0..16]
|A| [0..16, Enter - случайно]: 16
r - случайно, m - вручную, Enter - отмена: r
A = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A| = 16

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..16, Enter - случайно]: 0
r - случайно, m - вручную, Enter - отмена: r
B = {}, |B| = 0
```

Рисунок 14 — Граничные мощности: $A = U, B = \emptyset$

Ожидаемо $A \cup B = A \setminus B = A \triangle B = \bar{B} = A + B = A - B = U$, остальные результаты пусты (рисунки 15, 16).

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A ∪ B| = 16
A ∩ B = {}, |A ∩ B| = 0
A \ B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A \ B| = 16
B \ A = {}, |B \ A| = 0
A Δ B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A Δ B| = 16
¬A = {}, |¬A| = 0
¬B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |¬B| = 16
A + B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A + B| = 16
A - B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A - B| = 16
B - A = {}, |B - A| = 0
A * B = {}, |A * B| = 0
A / B = {}, |A / B| = 0
B / A = {}, |B / A| = 0
проверка: |¬A| = |U| - |A|: 0 = 16 - 16 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 16 + 0 = 16 + 0 верно
```

Рисунок 15 — Результаты операций при $A = U, B = \emptyset$

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход									
> m	000	001	011	010	110	111	101	100	·
U	2	2	2	2	2	2	2	2	16
A	2	2	2	2	2	2	2	2	16
B	0	0	0	0	0	0	0	0	0
A∪B	2	2	2	2	2	2	2	2	16
A∩B	0	0	0	0	0	0	0	0	0
A\B	2	2	2	2	2	2	2	2	16
B\A	0	0	0	0	0	0	0	0	0
AΔB	2	2	2	2	2	2	2	2	16
¬A	0	0	0	0	0	0	0	0	0
¬B	2	2	2	2	2	2	2	2	16
A+B	2	2	2	2	2	2	2	2	16
A-B	2	2	2	2	2	2	2	2	16
B-A	0	0	0	0	0	0	0	0	0
A*B	0	0	0	0	0	0	0	0	0
A/B	0	0	0	0	0	0	0	0	0
B/A	0	0	0	0	0	0	0	0	0

Рисунок 16 — Матрица кратностей при $A = U, B = \emptyset$

3.4 Сценарий 4: некорректный ввод команд и параметров

Команды a и o до создания универсума не выполняются (рисунок 17); «abc», «7», пустая строка, «uu», «-1» отвергаются, «U» принимается (рисунок 18).

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
! команда a недоступна: универсум ещё не создан (команда u)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
! команда o недоступна: универсум ещё не создан (команда u)

```

Рисунок 17 — Команды a и o до создания универсума

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> abc
! «abc» не команда; допустимы буквы u a b t o m h q
> 7
! «7» не команда; допустимы буквы u a b t o m h q
>
! пустая строка: введите одну из букв u a b t o m h q
> uu
! «uu» не команда; допустимы буквы u a b t o m h q
> -1
! «-1» не команда; допустимы буквы u a b t o m h q
> U

```

Рисунок 18 — Некорректный ввод команды

Разрядности 20, «-2», «abc» и кратности 0, 101 отвергаются; при $k = 1$ универсум — обычное множество (рисунок 19). Команда o называет оба незаданных мультимножества; способ «z» отвергается, пустая строка отменяет заполнение (рисунок 20).

```

n [0..19]: 20
! 20 не входит в [0..19]
n [0..19]: -2
! «-2» не целое неотрицательное число; нужно число из [0..19]
n [0..19]: abc
! «abc» не целое неотрицательное число; нужно число из [0..19]
n [0..19]: 3
k [1..100, Enter - случайно]: 0
! 0 не входит в [1..100]
k [1..100, Enter - случайно]: 101
! 101 не входит в [1..100]
k [1..100, Enter - случайно]: 1
код Грея,  $2^3 = 8$ : 000 001 011 010 110 111 101 100
 $U = \{000 \times 1, 001 \times 1, 011 \times 1, 010 \times 1, 110 \times 1, 111 \times 1, 101 \times 1, 100 \times 1\}$ ,  $|U| = 8$ 

```

Рисунок 19 — Некорректный ввод разрядности и кратности; универсум при $k = 1$

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
! команда o недоступна: не задано A (команда a) и B (команда b)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..8, Enter - случайно]: 3
r - случайно, m - вручную, Enter - отмена: z
! «z» не команда; допустимы буквы r m
r - случайно, m - вручную, Enter - отмена:
отменено

```

Рисунок 20 — Команда o до заполнения и отмена заполнения A

3.5 Сценарий 5: разрядность $n = 0$

$U = \{\}$, кратность не запрашивается (рисунок 21); допустима только мощность 0 (рисунок 22). Таблица содержит только строку мощностей, все результаты пусты, проверка выполняется (рисунки 23–25), что совпадает с подразделом 1.2.

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 0
при n = 0 в коде Грея нет слов, кратность не нужна
 $U = \{\}$ ,  $|U| = 0$ 

```

Рисунок 21 — Создание универсума при $n = 0$

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..0, Enter - случайно]: 1
! 1 не входит в [0..0]
|A| [0..0, Enter - случайно]: 0
r - случайно, m - вручную, Enter - отмена: m
A = {}, |A| = 0

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..0, Enter - случайно]:
|B| = 0
r - случайно, m - вручную, Enter - отмена: r
B = {}, |B| = 0

```

Рисунок 22 — Ручное и случайное заполнение при $n = 0$

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код   kU   kA   kB
Σ      0    0    0

```

Рисунок 23 — Таблица кратностей при $n = 0$

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {}, |A ∪ B| = 0
A ∩ B = {}, |A ∩ B| = 0
A \ B = {}, |A \ B| = 0
B \ A = {}, |B \ A| = 0
A Δ B = {}, |A Δ B| = 0
¬A      = {}, |¬A| = 0
¬B      = {}, |¬B| = 0
A + B = {}, |A + B| = 0
A - B = {}, |A - B| = 0
B - A = {}, |B - A| = 0
A * B = {}, |A * B| = 0
A / B = {}, |A / B| = 0
B / A = {}, |B / A| = 0
проверка: |¬A| = |U| - |A|: 0 = 0 - 0 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 0 + 0 = 0 + 0 верно

```

Рисунок 24 — Результаты всех операций при $n = 0$

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> m
      |·|
U      0
A      0
B      0
A∪B    0
A∩B    0
A\B    0
B\A    0
AΔB    0
¬A      0
¬B      0
A+B    0
A-B    0
B-A    0
A*B    0
A/B    0
B/A    0

```

Рисунок 25 — Матрица кратностей при $n = 0$

3.6 Сценарий 6: пересоздание универсума и конец ввода

После заполнения A и B при $n = 2$ (рисунок 26) создаётся универсум $n = 3$, и A, B сбрасываются (рисунок 27). Конец ввода во время ручного заполнения завершает программу с кодом 0 (рисунок 28).

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..12, Enter - случайно]: 4
r - случайно, m - вручную, Enter - отмена: r
A = {00×1, 01×1, 10×2}, |A| = 4

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..12, Enter - случайно]: 3
r - случайно, m - вручную, Enter - отмена: r
B = {00×2, 10×1}, |B| = 3

```

Рисунок 26 — Заполнение A и B над универсумом с $n = 2$

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 3
k [1..100, Enter - случайно]: 2
код Грея, 2^3 = 8: 000 001 011 010 110 111 101 100
U = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |U| = 16
A и B сброшены

```

Рисунок 27 — Пересоздание универсума с $n = 3$

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..16, Enter - случайно]: 2
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
A = {}, осталось 2:
ввод закончился

```

Рисунок 28 — Конец ввода во время ручного заполнения

3.7 Сценарий 7: наибольшая разрядность $n = 19$

$n = 19, k = 100$: $2^{19} = 524\,288$ слов, $|U| = 52\,428\,800$; выводятся первые 16 слов и сокращённая запись U (рисунок 29). $|A| = 4\,000\,752$ выбрана случайно (рисунок 30); таблица — первые 32 строки и полные мощности (рисунок 31). Все действия выполняются быстрее 1 с.

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 19
k [1..100, Enter - случайно]: 100
код Грея,  $2^{19} = 524288$ :
000000000000000000 00000000000000000001 00000000000000000011 00000000000000000010 000000000000000000110
00000000000000000111 00000000000000000101 00000000000000000100 000000000000000001100 000000000000000001101
000000000000000001111 000000000000000001110 000000000000000001010 000000000000000001011 000000000000000001001
000000000000000001000
... и ещё 524272
U = {00000000000000000000×100, 00000000000000000001×100, 00000000000000000011×100, 00000000000000000010×100, ... ещё 524284 эл.}, |U|
= 52428800

```

Рисунок 29 — Универсум при $n = 19, k = 100$

```

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..52428800, Enter - случайно]:
|A| = 4000752
r - случайно, m - вручную, Enter - отмена: r
A = {00000000000000000000×9, 00000000000000000001×8, 00000000000000000011×8, 00000000000000000010×13, ... ещё 524025 эл.}, |A| = 40007
52

```

Рисунок 30 — Случайное заполнение A при $n = 19$

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t

код	kU	kA	kB
000000000000000000	100	9	-
000000000000000001	100	8	-
000000000000000011	100	8	-
000000000000000010	100	13	-
0000000000000000110	100	4	-
0000000000000000111	100	4	-
0000000000000000101	100	7	-
0000000000000000100	100	6	-
00000000000000001100	100	4	-
00000000000000001101	100	10	-
00000000000000001111	100	7	-
00000000000000001110	100	14	-
00000000000000001010	100	6	-
00000000000000001011	100	6	-
00000000000000001001	100	6	-
00000000000000001000	100	9	-
000000000000000011000	100	7	-
000000000000000011001	100	10	-
000000000000000011011	100	8	-
000000000000000011010	100	6	-
000000000000000011110	100	14	-
000000000000000011111	100	8	-
000000000000000011101	100	9	-
000000000000000011100	100	8	-
000000000000000010100	100	11	-
000000000000000010101	100	8	-
000000000000000010111	100	8	-
000000000000000010110	100	11	-
000000000000000010010	100	12	-
000000000000000010011	100	4	-
000000000000000010001	100	2	-
000000000000000010000	100	5	-
... показаны 32 из 524288			
Σ	52428800	4000752	-

Рисунок 31 — Таблица кратностей при $n = 19$ (первые 32 строки)

ЗАКЛЮЧЕНИЕ

Разработана консольная программа на C++: генерация кода Грея разрядности 0–19, универсум с одинаковой кратностью k , ручное и автоматическое заполнение A и B заданной или случайной мощности, тринадцать операций с проверкой мощностей, таблица и матрица кратностей. Результаты совпали с ручным расчётом и с ожидаемыми значениями в граничных случаях ($A = U$, $B = \emptyset$, $n = 0$, $k = 1$); некорректный ввод отвергается с сообщением и повторным запросом.

Достоинства и недостатки.

- Код Грея строится за $O(2^n)$ без рекурсии; слово по номеру без предыдущих не получить.
- Мультимножество — вектор кратностей по номерам: операции за $O(2^n)$ без поиска, но память $O(2^n)$ даже для почти пустого.
- Случайное заполнение даёт точную мощность без холостых выборов; время $O(2^n + |A|)$ ограничивает $n_{\max} = 19$.
- Все элементы U имеют одну кратность; разные $k_U(x)$ задать нельзя.
- Команды — латинские буквы; при русской раскладке команда не распознаётся.

Масштабирование.

- Новая операция: метод `Multiset`, строка в `OPERATIONS` и вызов в `compute_operations`; справка и матрица подхватят её сами.
- Разные кратности в U : операции и `random_submultiset` уже работают с произвольными $k_U(x)$, нужна функция построения U из вектора кратностей.
- Больше двух мультимножеств: `fill_multiset` уже получает мультимножество и имя параметрами.
- Большие n : при $|A| > |U|/2$ строить A удалением единиц из U (не больше $|U|/2$ шагов).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Новиков, Ф. А. Дискретная математика для программистов : учебник для вузов / Ф. А. Новиков. — 3-е изд. — Санкт-Петербург : Питер, 2009. — 384 с.
- 2 Востров, А. В. Дискретная математика : курс лекций / А. В. Востров ; Санкт-Петербургский политехнический университет Петра Великого. — Санкт-Петербург, 2026. — URL: https://tema.spbstu.ru/dismath_lect/ (дата обращения: 04.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А

Текст программы

Листинг 3 — CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.10)
2 project(lab1 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 14)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 set(CORE_SOURCES
8     src/core/GrayCode.cpp
9     src/core/Random.cpp
10    src/core/Multiset.cpp
11    src/core/Universe.cpp
12 )
13
14 add_executable(lab1 src/main.cpp src/ui/Input.cpp src/ui/Menu.cpp
15    ${CORE_SOURCES})
16
17 # замер времени, по которому выбрана MAX_DEPTH
18 add_executable(bench tools/bench.cpp ${CORE_SOURCES})
19 target_include_directories(bench PRIVATE src)
```

Листинг 4 — src/core/Random.hpp

```
1 #pragma once
2 #include <cstdint>
3
4 // зерно для генератора: смесь std::random_device и текущего времени
5 unsigned make_seed();
6
7 // задаёт начальное состояние генератора; одно зерно — одна и та же
  последовательность чисел
8 void seed_random(unsigned seed);
9
10 // равновероятное целое число от 0 до bound - 1; bound должно быть больше
  нуля
11 size_t random_below(size_t bound);
```

Листинг 5 — src/core/Random.cpp

```

1 #include "core/Random.hpp"
2 #include <chrono>
3 #include <random>
4
5
6 // вихрь Мерсенна: период 2^19937 - 1, значения 32-битные (у std::rand всего
  15 бит)
7 static std::mt19937& generator(){
8     static std::mt19937 gen;
9     return gen;
10 }
11
12 unsigned make_seed(){
13     // в MinGW std::random_device детерминирован и при каждом запуске выдаёт
14     // одну и ту же последовательность, поэтому к нему примешивается время
15     std::random_device device;
16     unsigned time = static_cast<unsigned>(
17         std::chrono::high_resolution_clock::now().time_since_epoch().count()
18     );
19     return device() ^ time;
20 }
21 void seed_random(unsigned seed){
22     generator().seed(seed);
23 }
24
25 size_t random_below(size_t bound){
26     std::uniform_int_distribution<size_t> distribution(0, bound - 1);
27     return distribution(generator());
28 }

```

Листинг 6 — src/core/GrayCode.hpp

```

1 #pragma once
2 #include <cstdint>
3 #include <stdexcept>
4 #include <string>
5 #include <vector>
6
7 // наибольшая разрядность
8 const size_t MAX_DEPTH = 19;
9
10 // предел типа size_t
11 const size_t MAX_TYPE_DEPTH = sizeof(size_t) * 8 - 1;
12
13
14 class GrayCodeError : public std::runtime_error{

```

```

15 public:
16     explicit GrayCodeError(const std::string&
17         message):std::runtime_error(message){}
18 };
19
20 class InvalidDepthError : public GrayCodeError{
21 public:
22     InvalidDepthError():GrayCodeError("2^n не помещается в size_t"){ }
23 };
24
25 class CodeIndexError : public GrayCodeError{
26 public:
27     CodeIndexError():GrayCodeError("номер кода вне последовательности"){ }
28 };
29
30 // Генерация кода Грея
31 std::vector<size_t> generate_gray_code(size_t depth); // при depth = 0
32 // двоичная запись кода длиной depth, старший разряд слева
33 std::string code_to_string(size_t code, size_t depth);
34
35 // носитель мультимножеств
36 class CodeSet{
37     size_t depth;
38     std::vector<size_t> codes; // codes[i] — i-й код Грея
39     std::vector<size_t> positions; // positions[c] — номер кода c в
40     // последовательности
41 public:
42     explicit CodeSet(size_t depth);
43
44     size_t get_depth() const; // разрядность
45     size_t size() const; // число элементов
46     std::string get_code(size_t i) const;
47     bool find(const std::string& code, size_t& index) const;
48 };

```

Листинг 7 — src/core/GrayCode.cpp

```

1 #include "core/GrayCode.hpp"
2
3
4 // Функция, определяющая, какой бит перевернуть
5 static size_t changed_bit(size_t i){
6     size_t q = 0;
7     size_t j = i;

```

```

8     while(j % 2 == 0){
9         j /= 2;
10        q++;
11    }
12    // число нулей в конце двоичной записи i — номер бита справа, считая с
    нуля
13    return q;
14 }
15
16 std::vector<size_t> generate_gray_code(size_t depth){
17     /*
18         Алгоритм начинается с 0 и на каждом шаге переворачивает
19         один бит, какой бит перевернуть — решает функция changed_bit()
20     */
21     if(depth > MAX_TYPE_DEPTH){
22         throw InvalidDepthError();
23     }
24     std::vector<size_t> codes;
25     // при n = 0 кодов нет
26     if(depth == 0){
27         return codes;
28     }
29     size_t count = size_t(1) << depth;
30     codes.reserve(count);
31     size_t code = 0;
32     codes.push_back(code);
33     for(size_t i = 1; i < count; i++){
34         size_t p = changed_bit(i);
35         code ^= size_t(1) << p;
36         codes.push_back(code);
37     }
38     return codes;
39 }
40
41 // перевод кода Грея в строку
42 std::string code_to_string(size_t code, size_t depth){
43     std::string res;
44     for(size_t bit = depth; bit-- > 0;){
45         res += ((code >> bit) & 1) ? '1' : '0';
46     }
47     return res;
48 }
49
50
51 CodeSet::CodeSet(size_t
    depth):depth(depth), codes(generate_gray_code(depth)), positions(codes.size()){
52     for(size_t i = 0; i < codes.size(); i++){
53         positions[codes[i]] = i;
54     }

```

```

55 }
56
57 // получить длину слова
58 size_t CodeSet::get_depth() const{
59     return depth;
60 }
61
62 // получить размер вектора
63 size_t CodeSet::size() const{
64     return codes.size();
65 }
66
67 // получить код по индексу
68 std::string CodeSet::get_code(size_t i) const{
69     if(i >= codes.size()){
70         throw CodeIndexError();
71     }
72     return code_to_string(codes[i], depth);
73 }
74
75 // получить индекс по коду
76 bool CodeSet::find(const std::string& code, size_t& index) const{
77     if(codes.empty() || code.size() != depth){
78         return false;
79     }
80     size_t value = 0;
81     for(size_t i = 0; i < code.size(); i++){
82         if(code[i] != '0' && code[i] != '1') return false;
83         value = value * 2 + (code[i] - '0'); // превращает строку в число
84     }
85     // все  $2^n$  слов длины n встречаются в коде Грея, поэтому номер есть у
    любого value
86     index = positions[value];
87     return true;
88 }

```

Листинг 8 — src/core/Multiset.hpp

```

1 #pragma once
2 #include "core/GrayCode.hpp"
3 #include <cstdint>
4 #include <memory>
5 #include <ostream>
6 #include <stdexcept>
7 #include <string>
8 #include <vector>
9
10

```

```

11 class MultisetError : public std::runtime_error{
12 public:
13     explicit MultisetError(const std::string&
14         message):std::runtime_error(message){}
15 };
16
17 class MultisetIndexError : public MultisetError{
18 public:
19     MultisetIndexError():MultisetError("номер элемента вне носителя"){ }
20 };
21
22 class CarrierMismatchError : public MultisetError{
23 public:
24     CarrierMismatchError():MultisetError("мультимножества построены над
25         разными носителями"){ }
26 };
27
28 class NotSubmultisetError : public MultisetError{
29 public:
30     NotSubmultisetError():MultisetError("кратность элемента больше, чем в
31         универсуме"){ }
32 };
33
34 class CardinalityOverflowError : public MultisetError{
35 public:
36     CardinalityOverflowError():MultisetError("мощность больше мощности
37         ограничивающего мультимножества"){ }
38 };
39
40 // мультимножество над носителем — функция кратности: counts[i] = k(x_i), x_i
41 // — i-й код Грея;
42 // универсум U и мультимножества A, B — объекты этого класса над общим
43 // носителем
44 class Multiset{
45     std::shared_ptr<const CodeSet> carrier; // общий носитель
46     std::vector<size_t> counts; // кратности
47
48     // Сравнивает, одинаковый ли указатель у носителей
49     void check_same_carrier(const Multiset& other) const; // -
50     CarrierMismatchError
51     // доп проверка на подмножество
52     void check_within(const Multiset& universe) const; // -
53     NotSubmultisetError
54
55 public:
56     explicit Multiset(std::shared_ptr<const CodeSet> carrier); // все
57     кратности равны нулю
58
59 };

```

```

51     const CodeSet& get_carrier() const; // Ссылка на носитель
52     size_t size() const; // число элементов носителя
53     size_t get_count(size_t i) const; // получить кратность элемента
54     void set_count(size_t i, size_t count);
55     size_t cardinality() const; // сумма кратностей - мощность
56     bool is_submultiset_of(const Multiset& other) const;
57
58     // === операции ===
59
60     //теор. мн.
61     Multiset unite(const Multiset& b) const; // объединение
62     Multiset intersect(const Multiset& b) const; // пересечение
63     Multiset complement(const Multiset& universe) const; // дополнение
64     Multiset difference(const Multiset& b, const Multiset& universe) const;
        // разность
65     Multiset symmetric_difference(const Multiset& b, const Multiset&
        universe) const; // симметрическая разность
66
67     //арифм.
68     Multiset arithmetic_sum(const Multiset& b, const Multiset& universe)
        const; // сумма
69     Multiset arithmetic_difference(const Multiset& b) const; // разность
70     Multiset arithmetic_product(const Multiset& b, const Multiset& universe)
        const; // произведение
71     Multiset arithmetic_division(const Multiset& b) const; // деление
72
73     friend Multiset random_submultiset(const Multiset& bound, size_t
        cardinality); // автоматическое заполнение
74 };
75
76 // случайное подмультимножество bound мощности cardinality: cardinality раз
    случайный
77 // незаполненный элемент получает +1 к кратности
78 Multiset random_submultiset(const Multiset& bound, size_t cardinality);
79
80 // запись вида {000×2, 011×1}; элементы с нулевой кратностью не выводятся;
81 void print_multiset(std::ostream& out, const Multiset& multiset, size_t
    limit);
82 std::ostream& operator<<(std::ostream& out, const Multiset& multiset);

```

Листинг 9 — src/core/Multiset.cpp

```

1 #include "core/Multiset.hpp"
2 #include "core/Random.hpp"
3 #include <algorithm>
4
5

```

```

6 Multiset::Multiset(std::shared_ptr<const CodeSet>
  carrier):carrier(carrier),counts(carrier->size(), 0){}
7
8 void Multiset::check_same_carrier(const Multiset& other) const{
9     if(carrier != other.carrier){
10         throw CarrierMismatchError();
11     }
12 }
13
14 void Multiset::check_within(const Multiset& universe) const{
15     // носители сравнивает сам is_submultiset_of
16     if(!is_submultiset_of(universe)){
17         throw NotSubmultisetError();
18     }
19 }
20
21 const CodeSet& Multiset::get_carrier() const{
22     return *carrier;
23 }
24
25 size_t Multiset::size() const{
26     return counts.size();
27 }
28
29 size_t Multiset::get_count(size_t i) const{
30     if(i >= counts.size()){
31         throw MultisetIndexError();
32     }
33     return counts[i];
34 }
35
36 void Multiset::set_count(size_t i, size_t count){
37     if(i >= counts.size()){
38         throw MultisetIndexError();
39     }
40     counts[i] = count;
41 }
42
43 size_t Multiset::cardinality() const{
44     size_t res = 0;
45     for(size_t i = 0; i < counts.size(); i++){
46         res += counts[i];
47     }
48     return res;
49 }
50
51 bool Multiset::is_submultiset_of(const Multiset& other) const{
52     check_same_carrier(other);
53     for(size_t i = 0; i < counts.size(); i++){

```

```

54         if(counts[i] > other.counts[i]) return false;
55     }
56     return true;
57 }
58
59 Multiset Multiset::unite(const Multiset& b) const{
60     check_same_carrier(b);
61     Multiset res(carrier);
62     for(size_t i = 0; i < counts.size(); i++){
63         res.counts[i] = std::max(counts[i], b.counts[i]);
64     }
65     return res;
66 }
67
68 Multiset Multiset::intersect(const Multiset& b) const{
69     check_same_carrier(b);
70     Multiset res(carrier);
71     for(size_t i = 0; i < counts.size(); i++){
72         res.counts[i] = std::min(counts[i], b.counts[i]);
73     }
74     return res;
75 }
76
77 Multiset Multiset::complement(const Multiset& universe) const{
78     check_within(universe);
79     Multiset res(carrier);
80     for(size_t i = 0; i < counts.size(); i++){
81         res.counts[i] = universe.counts[i] - counts[i]; // гарантированно >=
            0, т.к. A подмножество U
82     }
83     return res;
84 }
85
86 Multiset Multiset::difference(const Multiset& b, const Multiset& universe)
const{
87     //  $A \setminus B = A \cap \neg B$ 
88     return intersect(b.complement(universe));
89 }
90
91 Multiset Multiset::symmetric_difference(const Multiset& b, const Multiset&
universe) const{
92     //  $A \triangle B = (A \setminus B) \cup (B \setminus A)$ 
93     return difference(b, universe).unite(b.difference(*this, universe));
94 }
95
96 Multiset Multiset::arithmetic_sum(const Multiset& b, const Multiset&
universe) const{
97     check_within(universe);
98     b.check_within(universe);

```

```

99     Multiset res(carrier);
100     for(size_t i = 0; i < counts.size(); i++){
101         res.counts[i] = std::min(counts[i] + b.counts[i],
102             universe.counts[i]);
103     }
104     return res;
105 }
106 Multiset Multiset::arithmetic_difference(const Multiset& b) const{
107     check_same_carrier(b);
108     Multiset res(carrier);
109     for(size_t i = 0; i < counts.size(); i++){
110         res.counts[i] = (counts[i] > b.counts[i]) ? counts[i] - b.counts[i] :
111             0;
112     }
113     return res;
114 }
115 Multiset Multiset::arithmetic_product(const Multiset& b, const Multiset&
116     universe) const{
117     check_within(universe);
118     b.check_within(universe);
119     Multiset res(carrier);
120     for(size_t i = 0; i < counts.size(); i++){
121         res.counts[i] = std::min(counts[i] * b.counts[i],
122             universe.counts[i]);
123     }
124     return res;
125 }
126 Multiset Multiset::arithmetic_division(const Multiset& b) const{
127     check_same_carrier(b);
128     Multiset res(carrier);
129     for(size_t i = 0; i < counts.size(); i++){
130         res.counts[i] = (b.counts[i] != 0) ? counts[i] / b.counts[i] : 0;
131     }
132     return res;
133 }
134
135 Multiset random_submultiset(const Multiset& bound, size_t cardinality){
136     if(cardinality > bound.cardinality()){
137         throw CardinalityOverflowError();
138     }
139     Multiset res(bound.carrier);
140     // номера элементов, кратность которых в res ещё меньше, чем в bound
141     std::vector<size_t> free;
142     free.reserve(bound.counts.size());
143     for(size_t i = 0; i < bound.counts.size(); i++){

```

```

144         if(bound.counts[i] > 0) free.push_back(i);
145     }
146     // cardinality раз: случайный элемент со свободным местом получает +1 к
кратности;
147     // список не пустеет раньше времени, так как свободных мест |bound| -
step > 0
148     for(size_t step = 0; step < cardinality; step++){
149         size_t j = random_below(free.size());
150         size_t i = free[j];
151         res.counts[i]++;
152         if(res.counts[i] == bound.counts[i]){
153             // элемент заполнен: на его место в списке встаёт последний,
список короче на 1
154             free[j] = free.back();
155             free.pop_back();
156         }
157     }
158     return res;
159 }
160
161
162 void print_multiset(std::ostream& out, const Multiset& multiset, size_t
limit){
163     const CodeSet& carrier = multiset.get_carrier();
164     size_t shown = 0;
165     size_t hidden = 0;
166     out << "{";
167     for(size_t i = 0; i < multiset.size(); i++){
168         size_t count = multiset.get_count(i);
169         if(count == 0) continue;
170         if(shown == limit){
171             hidden++;
172             continue;
173         }
174         if(shown > 0) out << ", ";
175         out << carrier.get_code(i) << "x" << count;
176         shown++;
177     }
178     if(hidden > 0) out << ", ... ещё " << hidden << " эл.";
179     out << "}";
180 }
181
182 std::ostream& operator<<(std::ostream& out, const Multiset& multiset){
183     print_multiset(out, multiset, multiset.size());
184     return out;
185 }

```

```

1 #pragma once
2 #include "core/GrayCode.hpp"
3 #include "core/Multiset.hpp"
4 #include <cstdint>
5 #include <memory>
6
7 // наибольшая кратность элемента универсума
8 const size_t MAX_MULTIPPLICITY = 100;
9
10
11 class InvalidMultiplicityError : public MultisetError{
12 public:
13     InvalidMultiplicityError():MultisetError("кратность элемента универсума
14     вне диапазона от 1 до MAX_MULTIPPLICITY"){ }
15 };
16
17 // универсум, в котором каждый код Грея входит с одной и той же кратностью
18 Multiset uniform_universe(std::shared_ptr<const CodeSet> carrier, size_t
multiplicity);

```

Листинг 11 — src/core/Universe.cpp

```

1 #include "core/Universe.hpp"
2
3
4 Multiset uniform_universe(std::shared_ptr<const CodeSet> carrier, size_t
multiplicity){
5     if(multiplicity < 1 || multiplicity > MAX_MULTIPPLICITY){
6         throw InvalidMultiplicityError();
7     }
8     Multiset universe(carrier);
9     for(size_t i = 0; i < universe.size(); i++){
10         universe.set_count(i, multiplicity);
11     }
12     return universe;
13 }

```

Листинг 12 — src/ui/Input.hpp

```

1 #pragma once
2 #include <cstdint>
3 #include <stdexcept>
4 #include <string>

```

```

5
6 // ввод закончился (Ctrl+Z в Windows, Ctrl+D в Linux, конец файла)
7 class InputClosedError : public std::runtime_error{
8 public:
9     InputClosedError():std::runtime_error("ввод закончился"){ }
10 };
11
12 // печатает приглашение и читает строку без пробелов, табуляций и \r по краям
13 std::string read_text(const std::string& prompt);
14
15 // строка из 1..9 цифр; число записывается в value
16 bool parse_number(const std::string& text, size_t& value);
17
18 // строка длины depth из символов 0 и 1
19 bool is_code(const std::string& text, size_t depth);
20
21 // переспрашивает, пока не будет введено целое число от min до max
22 size_t read_number(const std::string& prompt, size_t min, size_t max);
23
24 // то же, но пустая строка допустима: тогда возвращается false, value не
    меняется
25 bool read_number_or_empty(const std::string& prompt, size_t min, size_t max,
    size_t& value);
26
27 // переспрашивает, пока не будет введена одна буква из letters (регистр не
    важен);
28 // если allow_empty, пустая строка допустима и возвращается '\0'
29 char read_letter(const std::string& prompt, const std::string& letters, bool
    allow_empty);

```

Листинг 13 — src/ui/Input.cpp

```

1 #include "ui/Input.hpp"
2 #include <iostream>
3 #ifdef _WIN32
4 #include <io.h>
5 static bool stdin_is_console(){ return _isatty(0) != 0; }
6 #else
7 #include <unistd.h>
8 static bool stdin_is_console(){ return isatty(0) != 0; }
9 #endif
10
11
12 std::string read_text(const std::string& prompt){
13     std::cout << prompt;
14     std::string line;
15     if(!std::getline(std::cin, line)){

```

```

16         throw InputClosedError();
17     }
18     // при вводе из файла строка не отображается консолью, поэтому программа
    выводит её сама
19     static const bool echo = !stdin_is_console();
20     if(echo) std::cout << line.substr(0, line.find('\r')) << std::endl;
21     size_t begin = line.find_first_not_of(" \t\r");
22     if(begin == std::string::npos) return "";
23     size_t end = line.find_last_not_of(" \t\r");
24     return line.substr(begin, end - begin + 1);
25 }
26
27 // не больше 9 цифр: число до 999 999 999 гарантированно помещается в size_t
28 bool parse_number(const std::string& text, size_t& value){
29     if(text.empty() || text.size() > 9) return false;
30     size_t res = 0;
31     for(size_t i = 0; i < text.size(); i++){
32         if(text[i] < '0' || text[i] > '9') return false;
33         res = res * 10 + (text[i] - '0');
34     }
35     value = res;
36     return true;
37 }
38
39 bool is_code(const std::string& text, size_t depth){
40     if(text.size() != depth) return false;
41     for(size_t i = 0; i < text.size(); i++){
42         if(text[i] != '0' && text[i] != '1') return false;
43     }
44     return true;
45 }
46
47 // сообщение называет причину: пустая строка, не число или число вне отрезка
48 static void report_bad_number(const std::string& text, size_t min, size_t
max){
49     std::string range = "[" + std::to_string(min) + ".." +
    std::to_string(max) + "]";
50     size_t value = 0;
51     if(text.empty()){
52         std::cout << "! пустая строка: нужно целое число из " << range <<
    std::endl;
53     }
54     else if(parse_number(text, value)){
55         std::cout << "! " << value << " не входит в " << range << std::endl;
56     }
57     else{
58         std::cout << "! «" << text << "» не целое неотрицательное число;
    нужно число из " << range << std::endl;
59     }

```

```

60 }
61
62 size_t read_number(const std::string& prompt, size_t min, size_t max){
63     while(true){
64         std::string line = read_text(prompt);
65         size_t value = 0;
66         if(parse_number(line, value) && value >= min && value <= max){
67             return value;
68         }
69         report_bad_number(line, min, max);
70     }
71 }
72
73 bool read_number_or_empty(const std::string& prompt, size_t min, size_t max,
74 size_t& value){
75     while(true){
76         std::string line = read_text(prompt);
77         if(line.empty()) return false;
78         size_t res = 0;
79         if(parse_number(line, res) && res >= min && res <= max){
80             value = res;
81             return true;
82         }
83         report_bad_number(line, min, max);
84     }
85 }
86
87 char read_letter(const std::string& prompt, const std::string& letters, bool
88 allow_empty){
89     std::string listed;
90     for(size_t i = 0; i < letters.size(); i++){
91         listed += (i > 0 ? " " : "") + std::string(1, letters[i]);
92     }
93     while(true){
94         std::string line = read_text(prompt);
95         if(line.empty() && allow_empty) return '\0';
96         if(line.size() == 1){
97             char letter = line[0];
98             if(letter >= 'A' && letter <= 'Z') letter = letter - 'A' + 'a';
99             if(letters.find(letter) != std::string::npos) return letter;
100         }
101         if(line.empty()){
102             std::cout << "! пустая строка: введите одну из букв " << listed
103             << std::endl;
104         }
105         else{
106             std::cout << "! «" << line << "» не команда; допустимы буквы " <<
107             listed << std::endl;
108         }
109     }
110 }

```

```
105     }
106 }
```

Листинг 14 — src/ui/Menu.hpp

```
1  #pragma once
2  #include "core/GrayCode.hpp"
3  #include "core/Multiset.hpp"
4  #include <cstdint>
5  #include <memory>
6  #include <string>
7  #include <vector>
8
9  class Menu{
10     std::shared_ptr<const CodeSet> carrier;    // общий носитель U, A и B
11     Multiset universe;
12     Multiset a;
13     Multiset b;
14     bool universe_ready;
15     bool a_ready;
16     bool b_ready;
17
18     bool is_available(char command) const;
19
20     void create_universe();
21     void fill_multiset(Multiset& multiset, bool& ready, const std::string&
22         name);
23     bool fill_manual(Multiset& target, size_t cardinality, const std::string&
24         name) const;
25     bool parse_manual_line(const std::string& line, const Multiset& target,
26         size_t left,
27         std::vector<size_t>& indices, std::vector<size_t>&
28         counts) const;
29
30     std::vector<Multiset> compute_operations() const;
31     void show_help() const;
32     void show_table() const;
33     void show_operations() const;
34     void show_matrix() const;
35
36 public:
37     Menu();
38     void run();
39 };
```

Листинг 15 — src/ui/Menu.cpp

```

1 #include "ui/Menu.hpp"
2 #include "core/Random.hpp"
3 #include "core/Universe.hpp"
4 #include "ui/Input.hpp"
5 #include <algorithm>
6 #include <iostream>
7 #include <sstream>
8
9
10 static const size_t TABLE_LIMIT = 32;    // наибольшее число строк таблицы
11 static const size_t PRINT_LIMIT = 16;    // наибольшее число элементов
    мультимножества и слов кода в выводе
12 static const size_t LINE_WIDTH = 100;    // примерная наибольшая ширина
    записи мультимножества при  $n > 4$ 
13 static const size_t MATRIX_WIDTH = 112;  // наибольшая ширина строки матрицы
    кратностей
14
15 // операции в порядке вывода; результаты compute_operations идут в том же
    порядке
16 struct OperationInfo{
17     const char* title;    // название
18     const char* notation; // обозначение
19     const char* formula;  // кратность элемента в результате
20 };
21
22 static const OperationInfo OPERATIONS[] = {
23     {"объединение", "A  $\cup$  B", "max(kA, kB)"},
24     {"пересечение", "A  $\cap$  B", "min(kA, kB)"},
25     {"разность", "A  $\setminus$  B", "min(kA, kU - kB)"},
26     {"разность", "B  $\setminus$  A", "min(kB, kU - kA)"},
27     {"симметрическая разность", "A  $\triangle$  B", "max(k[A $\setminus$ B], k[B $\setminus$ A])"},
28     {"дополнение", " $\neg$ A", "kU - kA"},
29     {"дополнение", " $\neg$ B", "kU - kB"},
30     {"арифметическая сумма", "A + B", "min(kA + kB, kU)"},
31     {"арифметическая разность", "A - B", "max(kA - kB, 0)"},
32     {"арифметическая разность", "B - A", "max(kB - kA, 0)"},
33     {"арифметическое произведение", "A * B", "min(kA * kB, kU)"},
34     {"арифметическое деление", "A / B", "kA div kB; 0 при kB = 0"},
35     {"арифметическое деление", "B / A", "kB div kA; 0 при kA = 0"},
36 };
37 static const size_t OPERATION_COUNT = sizeof(OPERATIONS) /
    sizeof(OPERATIONS[0]);
38
39 static const char* COMMANDS = "u - универсум, a/b - задать A/B, t - таблица,
    о - операции, "
40
41                                     "m - матрица, h - справка, q - выход";
42

```

```

43 // ширина строки в символах: в UTF-8 продолжения символа имеют вид 10xxxxxx
44 static size_t text_width(const std::string& text){
45     size_t width = 0;
46     for(size_t i = 0; i < text.size(); i++){
47         if((static_cast<unsigned char>(text[i]) & 0xC0) != 0x80) width++;
48     }
49     return width;
50 }
51
52 static std::string pad_right(const std::string& text, size_t width){
53     size_t current = text_width(text);
54     return current >= width ? text : text + std::string(width - current, '
    ');
55 }
56
57 static std::string pad_left(const std::string& text, size_t width){
58     size_t current = text_width(text);
59     return current >= width ? text : std::string(width - current, ' ') +
    text;
60 }
61
62 // при n <= 4 запись полная (не больше 2^4 = PRINT_LIMIT элементов); при
    больших n элементов
63 // столько, сколько помещается примерно в LINE_WIDTH символов, но не меньше 4
64 static std::string format_multiset(const Multiset& multiset){
65     size_t depth = multiset.get_carrier().get_depth();
66     size_t limit = PRINT_LIMIT;
67     if(depth > 4){
68         size_t element_width = depth + 6;    // "код×100, "
69         limit = std::min(PRINT_LIMIT, std::max<size_t>(4, LINE_WIDTH /
    element_width));
70     }
71     std::ostringstream out;
72     print_multiset(out, multiset, limit);
73     return out.str();
74 }
75
76 // обозначение без пробелов для заголовков столбцов: A ∪ B -> AUB
77 static std::string compact(const std::string& notation){
78     std::string res = notation;
79     res.erase(std::remove(res.begin(), res.end(), ' '), res.end());
80     return res;
81 }
82
83
84 Menu::Menu():carrier(std::make_shared<CodeSet>(0)),universe(carrier),a(carri
    er),b(carrier),
85     universe_ready(false),a_ready(false),b_ready(false){}
86

```

```

87 void Menu::run() {
88     // список команд выводится перед каждым приглашением
89     while(true) {
90         std::cout << std::endl << COMMANDS << std::endl;
91         char command = read_letter("> ", "uabtomhq", false);
92         if(command == 'q') return;
93         if(!is_available(command)) continue;
94         switch(command) {
95             case 'u': create_universe(); break;
96             case 'a': fill_multiset(a, a_ready, "A"); break;
97             case 'b': fill_multiset(b, b_ready, "B"); break;
98             case 't': show_table(); break;
99             case 'o': show_operations(); break;
100            case 'm': show_matrix(); break;
101            case 'h': show_help(); break;
102        }
103    }
104 }
105
106 // печатает, чего не хватает для команды, и возвращает false, если команду
    ВЫПОЛНИТЬ НЕЛЬЗЯ
107 bool Menu::is_available(char command) const {
108     if(command == 'u' || command == 'h') return true;
109     if(!universe_ready) {
110         std::cout << "! команда " << command << " недоступна: универсум ещё
            не создан (команда u)" << std::endl;
111         return false;
112     }
113     if((command == 'o' || command == 'm') && (!a_ready || !b_ready)) {
114         std::string missing;
115         if(!a_ready) missing = "A (команда a)";
116         if(!b_ready) missing += std::string(missing.empty() ? "" : " и ") +
            "B (команда b)";
117         std::cout << "! команда " << command << " недоступна: не задано " <<
            missing << std::endl;
118         return false;
119     }
120     return true;
121 }
122
123 void Menu::create_universe() {
124     size_t depth = read_number("n [0.." + std::to_string(MAX_DEPTH) + "]: ",
        0, MAX_DEPTH);
125     std::shared_ptr<const CodeSet> new_carrier =
        std::make_shared<CodeSet>(depth);
126     size_t size = new_carrier->size();
127     Multiset new_universe(new_carrier);
128     size_t multiplicity = 0;
129

```

```

130     if(size == 0){
131         // элементов нет, поэтому и кратность задавать некому
132         std::cout << "при n = 0 в коде Грея нет слов, кратность не нужна" <<
            std::endl;
133     }
134     else{
135         if(!read_number_or_empty("k [1.." + std::to_string(MAX_MULTIPLICITY)
            + ", Enter - случайно]: ",
136                                 1, MAX_MULTIPLICITY, multiplicity)){
137             multiplicity = 1 + random_below(MAX_MULTIPLICITY);
138             std::cout << "k = " << multiplicity << std::endl;
139         }
140         new_universe = uniform_universe(new_carrier, multiplicity);
141     }
142
143     bool had_multisets = a_ready || b_ready;
144     carrier = new_carrier;
145     universe = new_universe;
146     a = Multiset(carrier);
147     b = Multiset(carrier);
148     universe_ready = true;
149     a_ready = false;
150     b_ready = false;
151
152     // не больше PRINT_LIMIT слов кода, в строке столько, сколько помещается
    в LINE_WIDTH
153     if(size > 0){
154         size_t shown = std::min(size, PRINT_LIMIT);
155         size_t per_line = std::max<size_t>(1, LINE_WIDTH / (depth + 1));
156         std::cout << "код Грея, 2^" << depth << " = " << size << ":";
157         for(size_t i = 0; i < shown; i++){
158             if(shown > per_line && i % per_line == 0) std::cout << std::endl;
159             else if(i > 0 || shown <= per_line) std::cout << " ";
160             std::cout << carrier->get_code(i);
161         }
162         std::cout << std::endl;
163         if(shown < size) std::cout << "... и ещё " << size - shown <<
            std::endl;
164     }
165     std::cout << "U = " << format_multiset(universe) << ", |U| = " <<
        universe.cardinality() << std::endl;
166     if(had_multisets) std::cout << "A и B сброшены" << std::endl;
167 }
168
169 void Menu::fill_multiset(Multiset& multiset, bool& ready, const std::string&
    name){
170     size_t high = universe.cardinality();
171     size_t cardinality = 0;

```

```

172     if(!read_number_or_empty("|" + name + "| [0.." + std::to_string(high) +
173         ", Enter - случайно]: ",
174             0, high, cardinality)){
175         cardinality = random_below(high + 1);
176         std::cout << "|" << name << "| = " << cardinality << std::endl;
177     }
178     char method = read_letter("r - случайно, m - вручную, Enter - отмена: ",
179         "rm", true);
180     if(method == '\0'){
181         std::cout << "отменено" << std::endl;
182         return;
183     }
184     // заполняется копия: при отмене ручного ввода прежнее мультимножество
185     // сохраняется
186     Multiset filled(carrier);
187     if(method == 'm'){
188         if(!fill_manual(filled, cardinality, name)){
189             std::cout << "отменено" << std::endl;
190             return;
191         }
192     }
193     else{
194         filled = random_submultiset(universe, cardinality);
195     }
196     multiset = filled;
197     ready = true;
198     std::cout << name << " = " << format_multiset(multiset) << ", |" << name
199     << " | = "
200     << multiset.cardinality() << std::endl;
201 }
202
203 bool Menu::fill_manual(Multiset& target, size_t cardinality, const
204     std::string& name) const{
205     if(cardinality == 0) return true;
206     std::cout << "пары код:кратность через пробел (01:2 11), каждого кода не
207     больше "
208     << universe.get_count(0) << ", Enter - отмена" << std::endl;
209     while(target.cardinality() < cardinality){
210         size_t left = cardinality - target.cardinality();
211         std::string line = read_text(name + " = " + format_multiset(target) +
212             ", осталось " + std::to_string(left) + ": ");
213         if(line.empty()) return false;
214         std::vector<size_t> indices;
215         std::vector<size_t> counts;
216         // строка применяется только целиком: при ошибке в любой паре A не
217         // меняется
218         if(parse_manual_line(line, target, left, indices, counts)){

```

```

213         for(size_t i = 0; i < indices.size(); i++){
214             target.set_count(indices[i], target.get_count(indices[i]) +
                counts[i]);
215         }
216     }
217 }
218 return true;
219 }
220
221 // разбирает строку пар код:кратность; при ошибке печатает её причину и
    возвращает false
222 bool Menu::parse_manual_line(const std::string& line, const Multiset& target,
    size_t left,
223                             std::vector<size_t>& indices,
                                std::vector<size_t>& counts) const{
224     size_t depth = carrier->get_depth();
225     size_t total = 0;
226     std::istringstream words(line);
227     std::string word;
228     while(words >> word){
229         size_t colon = word.find(':');
230         std::string code = word.substr(0, colon);
231         size_t count = 1;
232         if(!is_code(code, depth)){
233             std::cout << "! «" << word << "»: код должен состоять из " <<
                depth << " символов 0 и 1" << std::endl;
234             return false;
235         }
236         if(colon != std::string::npos && (!parse_number(word.substr(colon +
            1), count) || count == 0)){
237             std::cout << "! «" << word << "»: после двоеточия нужна кратность
                - целое число от 1" << std::endl;
238             return false;
239         }
240         size_t index = 0;
241         carrier->find(code, index);    // код длины n из 0 и 1 всегда есть в
            носителе
242         // учитываются и единицы этого же кода, уже записанные в строке
            раньше
243         size_t taken = target.get_count(index);
244         for(size_t i = 0; i < indices.size(); i++){
245             if(indices[i] == index) taken += counts[i];
246         }
247         size_t free = universe.get_count(index) - std::min(taken,
            universe.get_count(index));
248         if(count > free){
249             std::cout << "! «" << word << "»: у " << code << " свободно " <<
                free << " (kU = "

```

```

250         << universe.get_count(index) << ", занято " << taken <<
           << ")" << std::endl;
251     return false;
252 }
253 indices.push_back(index);
254 counts.push_back(count);
255 total += count;
256 }
257 if(total > left){
258     std::cout << "! в строке " << total << " ед., а осталось распределить
           << " << left << std::endl;
259     return false;
260 }
261 return true;
262 }
263
264 std::vector<Multiset> Menu::compute_operations() const{
265     std::vector<Multiset> res;
266     res.push_back(a.unite(b));
267     res.push_back(a.intersect(b));
268     res.push_back(a.difference(b, universe));
269     res.push_back(b.difference(a, universe));
270     res.push_back(a.symmetric_difference(b, universe));
271     res.push_back(a.complement(universe));
272     res.push_back(b.complement(universe));
273     res.push_back(a.arithmetic_sum(b, universe));
274     res.push_back(a.arithmetic_difference(b));
275     res.push_back(b.arithmetic_difference(a));
276     res.push_back(a.arithmetic_product(b, universe));
277     res.push_back(a.arithmetic_division(b));
278     res.push_back(b.arithmetic_division(a));
279     return res;
280 }
281
282 void Menu::show_help() const{
283     std::cout << "запись {000×2, 011×1}: код Грея × кратность, {} - пустое;
           kU, kA, kB - кратности в U, A, B" << std::endl;
284     for(size_t i = 0; i < OPERATION_COUNT; i++){
285         std::cout << pad_right(OPERATIONS[i].notation, 7) <<
           pad_right(OPERATIONS[i].formula, 26)
286             << OPERATIONS[i].title << std::endl;
287     }
288     std::cout << "0 ≤ n ≤ " << MAX_DEPTH << ", 1 ≤ k ≤ " <<
           MAX_MULTIPLICITY << ", 0 ≤ |A|, |B| ≤ |U|" << std::endl;
289 }
290
291 void Menu::show_table() const{
292     size_t code_width = std::max<size_t>(carrier->get_depth(), 3);

```

```

293 // числа выравниваются по самому длинному из них - мощности |U| в строке
    Σ
294 size_t number_width =
    std::max<size_t>(std::to_string(universe.cardinality()).size(), 3) + 1;
295 std::cout << pad_right("код", code_width) << pad_left("kU", number_width)
296         << pad_left("kA", number_width) << pad_left("kB", number_width)
        << std::endl;
297 size_t rows = std::min(universe.size(), TABLE_LIMIT);
298 for(size_t i = 0; i < rows; i++){
299     std::cout << pad_right(carrier->get_code(i), code_width)
300         << pad_left(std::to_string(universe.get_count(i)),
        number_width)
301         << pad_left(a_ready ? std::to_string(a.get_count(i)) : "-",
        number_width)
302         << pad_left(b_ready ? std::to_string(b.get_count(i)) : "-",
        number_width) << std::endl;
303 }
304 if(rows < universe.size()){
305     std::cout << "... показаны " << rows << " из " << universe.size() <<
        std::endl;
306 }
307 std::cout << pad_right("Σ", code_width)
308         << pad_left(std::to_string(universe.cardinality()),
        number_width)
309         << pad_left(a_ready ? std::to_string(a.cardinality()) : "-",
        number_width)
310         << pad_left(b_ready ? std::to_string(b.cardinality()) : "-",
        number_width) << std::endl;
311 }
312
313 void Menu::show_operations() const{
314     std::vector<Multiset> results = compute_operations();
315     for(size_t i = 0; i < OPERATION_COUNT; i++){
316         std::cout << pad_right(OPERATIONS[i].notation, 6) << "= " <<
        format_multiset(results[i])
317         << ", |" << OPERATIONS[i].notation << "| = " <<
        results[i].cardinality() << std::endl;
318     }
319
320 // следствия определений:  $|\neg A| = |U| - |A|$  и  $|A \cup B| + |A \cap B| = |A| + |B|$ 
321 size_t u = universe.cardinality();
322 size_t na = a.cardinality();
323 size_t nb = b.cardinality();
324 size_t complement = results[5].cardinality();
325 size_t unite = results[0].cardinality();
326 size_t intersect = results[1].cardinality();
327 std::cout << "проверка:  $|\neg A| = |U| - |A|$ : " << complement << " = " << u
    << " - " << na

```

```

328         << (complement == u - na ? " верно" : " НЕВЕРНО") << std::endl;
329     std::cout << "проверка:  $|A \cup B| + |A \cap B| = |A| + |B|$ : " << unite << " +
        " << intersect << " = " << na << " + " << nb
330         << (unite + intersect == na + nb ? " верно" : " НЕВЕРНО") <<
        std::endl;
331 }
332
333 void Menu::show_matrix() const{
334     std::vector<Multiset> results = compute_operations();
335     std::vector<std::string> titles = {"U", "A", "B"};
336     std::vector<const Multiset*> rows = {&universe, &a, &b};
337     for(size_t i = 0; i < OPERATION_COUNT; i++){
338         titles.push_back(compact(OPERATIONS[i].notation));
339         rows.push_back(&results[i]);
340     }
341
342     // столбцов столько, сколько помещается в MATRIX_WIDTH
343     const size_t label_width = 4;
344     const size_t total_width = 9;
345     size_t column_width = std::max<size_t>(carrier->get_depth(), 3) + 2;
346     size_t columns = std::min(universe.size(), (MATRIX_WIDTH - label_width -
        total_width) / column_width);
347
348     std::cout << pad_right("", label_width);
349     for(size_t j = 0; j < columns; j++) std::cout <<
        pad_left(carrier->get_code(j), column_width);
350     std::cout << pad_left("|·|", total_width) << std::endl;
351     for(size_t i = 0; i < rows.size(); i++){
352         std::cout << pad_right(titles[i], label_width);
353         for(size_t j = 0; j < columns; j++){
354             std::cout << pad_left(std::to_string(rows[i]->get_count(j)),
                column_width);
355         }
356         std::cout << pad_left(std::to_string(rows[i]->cardinality()),
            total_width) << std::endl;
357     }
358     if(columns < universe.size()){
359         std::cout << "... показаны " << columns << " из " << universe.size()
            << " кодов, мощности по всем" << std::endl;
360     }
361 }

```

Листинг 16 — src/main.cpp

```

1 #include "core/Random.hpp"
2 #include "ui/Input.hpp"
3 #include "ui/Menu.hpp"

```

```

4 #include <cstdlib>
5 #include <cstring>
6 #include <exception>
7 #include <iostream>
8 #ifdef _WIN32
9 #include <windows.h>
10 #endif
11
12
13 int main(int argc, char* argv[]){
14 #ifdef _WIN32
15     // строки программы записаны в UTF-8, а консоль Windows по умолчанию
16     // выводит в кодировке 866
17     SetConsoleOutputCP(CP_UTF8);
18 #endif
19     // lab1 --seed N повторяет запуск с теми же случайными числами
20     unsigned seed = make_seed();
21     if(argc == 3 && std::strcmp(argv[1], "--seed") == 0){
22         seed = static_cast<unsigned>(std::strtoul(argv[2], nullptr, 10));
23     }
24     seed_random(seed);
25     std::cout << "seed = " << seed << std::endl;
26
27     try{
28         Menu menu;
29         menu.run();
30     }
31     catch(const InputClosedError&){
32         std::cout << std::endl << "ввод закончился" << std::endl;
33     }
34     catch(const std::exception& error){
35         std::cout << "Внутренняя ошибка: " << error.what() << std::endl;
36         return 1;
37     }
38     return 0;
39 }

```

Листинг 17 — tools/bench.cpp

```

1 // Замер времени самых долгих действий программы при разных разрядностях n.
2 // По нему выбрана MAX_DEPTH: наибольшее n, при котором любое действие
3 // занимает меньше 1 с.
4 // Сборка: g++ -std=c++14 -O2 -Isrc tools/bench.cpp src/core/*.cpp -o bench
5 #include "core/GrayCode.hpp"
6 #include "core/Multiset.hpp"
7 #include "core/Random.hpp"
8 #include "core/Universe.hpp"

```

```

8 #include <algorithm>
9 #include <chrono>
10 #include <iomanip>
11 #include <iostream>
12 #include <memory>
13
14
15 // наименьшее из трёх измерений, мс: так меньше сказываются посторонние
    процессы
16 template<class Action>
17 static double measure(Action action){
18     double best = 1e18;
19     for(int run = 0; run < 3; run++){
20         auto start = std::chrono::steady_clock::now();
21         action();
22         auto finish = std::chrono::steady_clock::now();
23         best = std::min(best, std::chrono::duration<double,
                std::milli>(finish - start).count());
24     }
25     return best;
26 }
27
28 int main(){
29     seed_random(1);
30     std::cout << " n    код Грея    универсум    заполнение A    13 операций
        наибольшее, мс" << std::endl;
31     for(size_t depth = 10; depth <= MAX_DEPTH + 2; depth++){
32         std::shared_ptr<const CodeSet> carrier;
33         double t_code = measure([&]{ carrier =
                std::make_shared<CodeSet>(depth); });
34
35         // наихудший случай: все кратности равны MAX_MULTIPLICITY
36         Multiset universe(carrier);
37         double t_universe = measure([&]{ universe = uniform_universe(carrier,
                MAX_MULTIPLICITY); });
38
39         // заполнение делает |A| шагов, поэтому дольше всего при |A| = |U|
40         Multiset a(carrier);
41         Multiset b(carrier);
42         double t_fill = measure([&]{ a = random_submultiset(universe,
                universe.cardinality()); });
43         a = random_submultiset(universe, universe.cardinality() / 2);
44         b = random_submultiset(universe, universe.cardinality() / 3);
45
46         size_t checksum = 0;
47         double t_operations = measure([&]{
48             checksum += a.unite(b).cardinality() +
                a.intersect(b).cardinality()

```

```

49         + a.difference(b, universe).cardinality() +
        b.difference(a, universe).cardinality()
50     + a.symmetric_difference(b, universe).cardinality()
51     + a.complement(universe).cardinality() +
        b.complement(universe).cardinality()
52     + a.arithmetic_sum(b, universe).cardinality() +
        a.arithmetic_difference(b).cardinality()
53     + b.arithmetic_difference(a).cardinality() +
        a.arithmetic_product(b, universe).cardinality()
54     + a.arithmetic_division(b).cardinality() +
        b.arithmetic_division(a).cardinality();
55     });
56
57     double worst = std::max({t_code, t_universe, t_fill, t_operations});
58     std::cout << std::fixed << std::setprecision(1) << std::setw(2) <<
    depth
59         << std::setw(11) << t_code << std::setw(12) << t_universe
        << std::setw(15) << t_fill
60         << std::setw(14) << t_operations << std::setw(17) << worst
61         << (worst < 1000 ? "" : " > 1 c") << std::endl;
62     if(checksum == 0) std::cout << "";    // не даёт компилятору выбросить
        операции
63 }
64 return 0;
65 }

```